

SecureOrg Inc.

PCI DSS Compliance Framework

Secure Development Policy

Document Reference:	SDP-010
Version:	1.4
Effective Date:	April 14, 2026
Document Owner:	Chief Technology Officer
Classification:	CONFIDENTIAL
Review Cycle:	Annual
Framework:	PCI DSS v4.0

This document contains confidential and proprietary information. Unauthorized disclosure is prohibited.

1. Purpose and Scope

1.1 Purpose

This Secure Development Policy defines the security requirements that must be embedded into every phase of the software development lifecycle (SDLC) for all applications and systems developed by or for SecureOrg Inc. that interact with the cardholder data environment. The Policy implements PCI DSS v4.0 Requirement 6 (Develop and Maintain Secure Systems and Software), ensuring that security is built into applications by design rather than added as an afterthought.

1.2 Scope

This Policy applies to all in-house developed software, customized third-party software, and infrastructure-as-code used in or connected to the CDE, regardless of development methodology (waterfall, agile, DevOps).

2. Secure SDLC Framework

2.1 Security Integration by Phase

Requirements	Define security and compliance requirements; threat scenarios	Security requirements document
Design	Threat modeling; secure architecture review; data flow diagrams	Threat model; architecture review sign-off
Development	Secure coding standards; peer code review; SAST	Code review records; SAST scan results
Testing	DAST; penetration testing; vulnerability scanning	Test reports; remediation evidence
Deployment	Change control; security sign-off; configuration review	Change record; CISO/security sign-off
Operations	Continuous monitoring; patch management; periodic review	Monitoring dashboards; patch records
Decommission	Secure data destruction; documentation archival	Decommission checklist; disposal records

2.2 Threat Modeling

Formal threat modeling shall be conducted for all new applications and significant changes to existing applications in the CDE. Threat modeling shall:

- Identify and document all assets, data flows, and entry points
- Identify and prioritize threats using STRIDE or equivalent methodology
- Define mitigating controls for each identified threat
- Be reviewed and approved by the Information Security team
- Be updated when significant changes are made to the application architecture

3. Secure Coding Standards

3.1 General Secure Coding Requirements

All developers must adhere to the following secure coding principles:

- Validate all input: never trust data from external sources; implement server-side validation
- Implement output encoding appropriate to the context (HTML, SQL, JavaScript, etc.)
- Use parameterized queries or prepared statements for all database interactions; never construct SQL from user input

- Implement proper error handling: display generic error messages to users; log detailed errors server-side
- Never hardcode credentials, API keys, or secrets in source code
- Use cryptographic functions from approved libraries; never implement custom cryptography
- Implement secure session management per OWASP Session Management guidelines

3.2 OWASP Top 10 Mitigation

Applications processing CHD must be developed with controls addressing the OWASP Top 10 risks including:

- Broken Access Control: Implement access controls at the server side; enforce least privilege
- Cryptographic Failures: Use only approved algorithms for CHD (see Data Handling Policy DHEP-006)
- Injection: Parameterize all queries; whitelist input validation
- Security Misconfiguration: Automated security configuration checks in CI/CD pipeline
- Vulnerable Components: Software composition analysis (SCA) in build pipeline; maintain component inventory
- Identification and Authentication Failures: Implement strong MFA; secure session management
- Security Logging and Monitoring Failures: Log all authentication events and access to CHD
- SSRF: Validate and restrict server-side requests; whitelist allowed external destinations

3.3 CHD-Specific Coding Requirements

- PAN must never be written to logs, temporary files, or debug output
- Display of PAN must implement masking at the presentation layer (first 6 / last 4 maximum)
- SAD must never be stored post-authorization; implementing code must explicitly clear SAD from memory
- Cryptographic key handling code must use approved key management libraries
- Payment processing code must be reviewed by a PCI DSS-knowledgeable security reviewer

4. Code Review Requirements

4.1 Mandatory Code Review

Code review is mandatory for all changes deployed to production CDE systems. The review process shall:

- Be conducted by a developer other than the code author
- Use a formal code review checklist that includes security-specific items
- Be documented with reviewer identity and sign-off date
- Verify that the code does not introduce vulnerabilities in the OWASP Top 10 categories
- Confirm secure handling of CHD and absence of hardcoded credentials

4.2 Automated Security Testing

The CI/CD pipeline shall integrate the following automated security checks:

- Static Application Security Testing (SAST): Scan every code commit for security vulnerabilities
- Software Composition Analysis (SCA): Identify known vulnerabilities in third-party libraries (CVE database)
- Secret scanning: Detect hardcoded credentials, API keys, and other secrets in code
- Infrastructure-as-Code scanning: Detect security misconfigurations in IaC templates

Build pipelines must fail and block deployment if SAST or SCA findings of Critical or High severity are not resolved or formally accepted through the risk management process.

5. Test Environment Security

5.1 Separation of Environments

- Development, testing/staging, and production environments must be physically or logically separated
- Developers must not have direct access to production CDE environments
- Production CHD must not be used in development or test environments under any circumstances
- Where test environments require CHD-like data, approved data masking or synthetic data generation tools must be used

5.2 Test Data Requirements

Test data management is critical to preventing CHD exposure in non-production environments:

- All production CHD must be obfuscated, tokenized, or replaced with synthetic data before use in testing
- Developers and testers with access to obfuscated test data must still comply with data handling requirements
- Test environments must be scanned for real CHD at least quarterly
- Discovery of real CHD in test environments is treated as a security incident

6. Third-Party and Open Source Components

6.1 Component Inventory

All third-party software components and open source libraries used in CDE applications must be:

- Inventoried in a software bill of materials (SBOM) maintained for each application
- Obtained only from trusted, official sources (official repositories, vendor portals)
- Verified for integrity before use (cryptographic hash verification)
- Assessed for known vulnerabilities using SCA tools before adoption and continuously thereafter

6.2 Component Vulnerability Management

- Critical and High severity CVEs in active components must be remediated within 30 days
- Abandoned or end-of-life libraries must be replaced within 90 days of EOL announcement
- The SBOM must be reviewed at every major release and at least quarterly for active applications

6.3 PCI DSS v4.0 Requirement 6.3 — Web Applications

All public-facing web applications in scope for PCI DSS must be protected by one of the following:

- A Web Application Firewall (WAF) actively blocking OWASP Top 10 attack categories, with rules reviewed and updated at least monthly
- Annual penetration testing conducted by a qualified application security assessor

7. Revision History

1.0	Jan 15, 2024	Initial release	CTO Office
1.2	Aug 12, 2024	Added CI/CD pipeline security; SBOM requirements per PCI DSS v4.0 Req 6	Dev/Security
1.4	Apr 14, 2026	Annual review; threat modeling requirements; secret scanning	CTO/CISO